

MedNote Scribe: Request-to-Response Pipeline Explained

Overview

When a doctor types something into MedNote Scribe, the system doesn't just send it straight to an LLM. It goes through a carefully orchestrated pipeline ? each step exists for a specific reason. Think of it like a hospital triage system: the request gets classified, routed to the right department, processed with the right tools, checked for safety, and then delivered back.

```
graph TD
  A[???? Doctor types input] --> B[Step 1: Parse Input]
  B --> C[What does the doctor want?]
  C -->|Generate a note| D[Step 2: Context Extraction]
  C -->|Look up an ICD code| E[Step 3: Entity Extraction]
  C -->|Save to chart| F[Step 7: Tool Execution]
  C -->|Check history| G[Step 8: Memory Lookup]
  C -->|Asking for diagnosis| H[Refusal Response]
  D --> E
  E --> I[Step 4: RAG Pipeline]
  I -->|SOAP note| J[Step 5: Note Generation]
  I -->|Code lookup| K[Step 6: Response Generation]
  J --> L[Step 6: Guardrail Check]
  L -->|Safe| K
  L -->|Dangerous| M[Escalation Warning]
  F --> K
  G --> K
  M --> K
  K --> N[? Doctor sees the response]
```

Step 1: Parse Input (Intent Classification)

What it does

Reads the doctor's message and figures out what they're asking for.

Why it's needed

The system handles 5 completely different types of requests. Without classification, it wouldn't know whether to generate a SOAP note, look up a code, save something, or refuse a request. Each type follows a different path through the system.

How it works

- 1. Checks for keywords in the input:
 - ? "save", "chart", "store" ? Save intent (route to tools)
 - ? "ICD", "code", "coding" ? ICD lookup intent (route to RAG only)
 - ? "history", "last visit", "prior" ? History intent (route to memory)
 - ? "diagnose", "diagnosis" ? Refuse intent (safety guardrail)
 - ? Everything else ? SOAP note intent (full pipeline)
- 2. For ambiguous inputs, a fast LLM classifier makes the final call.

Example

```
Input: "Patient reports headache for 3 days, worse in the morning. BP 130/85."  
? No save/ICD/history/diagnose keywords found  
? Intent: "soap" ? Route to full SOAP generation pipeline
```

Step 2: Context Extraction (Patient Demographics)

What it does

Pulls the patient's age and biological sex from the mock EHR system.

Why it's needed

- ICD-10 codes are sex- and age-specific. For example:
- ? Pregnancy codes (O00?O9A) only apply to female patients
 - ? Prostate codes (N40?N42) only apply to male patients
 - ? Perinatal codes (P00?P96) only apply to newborns (?28 days)

Without this step, the system might suggest pregnancy codes for a male patient ? medically nonsensical and embarrassing in a demo.

How it works

```
Patient ID: P001 ? Query mock EHR ? {age: 34, sex: "female"}
```

These demographics become a hard filter on the vector database query later.

Step 3: Entity Extraction (Clinical NLP)

What it does

Uses a fast, cheap LLM to extract standard clinical terms from the doctor's input.

Why it's needed

Doctors and patients speak in everyday language. The vector database contains formal medical terminology. This step bridges the gap:

What the doctor/patient says	What the system searches for
"ear infection in both ears"	"Acute bilateral otitis media"
"blood pressure is running high"	"Essential hypertension"
"can't breathe, chest is tight"	"Acute dyspnea"
"sugar is high"	"Type 2 diabetes mellitus"

Why not just search with the raw transcript?

- ? Transcripts contain noise: greetings, scheduling talk, tangents
- ? A 5-minute transcript might be 2,000 words, but only 2?3 clinical conditions matter

? Targeted clinical terms produce dramatically better search results than dumping raw conversation into a vector query

How it works

```
Input: "Looks like the kid has an ear infection in both ears,
       and the mom's blood pressure is running high"

Prompt ? Fast LLM (Claude Haiku / Gemini Flash)

Output: ["Acute bilateral otitis media", "Essential hypertension"]
```

These extracted entities become the search queries for the RAG pipeline.

Step 4: RAG Pipeline (The Core Intelligence)

What is RAG?

RAG = Retrieval-Augmented Generation

Instead of asking the LLM to recall ICD-10 codes from memory (where it might hallucinate fake codes), we:

- 1. Retrieve the correct codes from a verified database
- 2. Augment the LLM's prompt with those real codes
- 3. Let the LLM Generate a response grounded in real data

Think of it like this: instead of asking a student to answer from memory (risky), you give them the textbook open to the right page and say "use this."

Why RAG matters here

- ? There are ~72,000 ICD-10 codes ? no LLM has perfectly memorized all of them
- ? A hallucinated code (one that doesn't exist) could lead to billing fraud
- ? Every suggestion must be traceable back to the official source for audits
- ? RAG provides the citation ("Source: ICD-10-CM 2026") that makes the system trustworthy

Where the Data Comes From (The ETL Pipeline)

Before RAG can work, we need to build the knowledge base from the official ICD-10-CM 2026 XML files. This is a one-time ETL (Extract-Transform-Load) step that runs before the system goes live.

We use two source files, each serving a different purpose:

Source File	Size	Purpose
icd10cm_tabular_2026.xml	9.3 MB (243K lines)	The authority ? full code definitions, hierarch...
icd10cm_index_2026.xml	9.2 MB (322K lines)	The synonym dictionary ? maps natural-language ...

Why two files?

The Tabular file tells the system what each code means:

```
Hierarchy: Diseases of ear ? Otitis media
Includes: serous otitis media NOS
Excludes1: acute serous otitis media (H65.0-)
```

H65.

The Index file tells the system what each condition is called in everyday language:

```
??? external ? H60.9
??? middle ? H66.9
??? bilateral ? H65.93
```

This is critical because doctors say "ear infection," not "H65.93." The Index provides a pre-built synonym bridge between colloquial language and formal codes ? the same bridge that Entity Extraction (Step 3) and the SapBERT embedding model try to create. Having the Index's synonyms baked directly into the embedding text means the vector search can find codes even when SapBERT alone wouldn't have mapped the term correctly.

What gets embedded

Each of the ~72,000 ICD-10 codes becomes a single document with all synonym sources merged:

```
H65.93: Unspecified otitis media, bilateral
Hierarchy: Diseases of ear ? Nonsuppurative otitis media ? Unspecified otitis media
Also known as: serous otitis media NOS, Ear infection bilateral, Middle ear infection both ears
Excludes: acute serous otitis media (H65.0-)
```

The "Also known as" line combines three sources:

- 1. from the Tabular XML (official synonyms)
- 2. from the Tabular XML ("also known as" names)
- 3. Index synonyms parsed from the Index XML (natural-language terms)

This gives the SapBERT embedding model maximum signal for mapping any phrasing of a condition to the right code.

The 5 Sub-Steps of the RAG Pipeline

```
graph LR
  A["3.1<br/>Context Filter<br/><i>age, sex</i>"] --> B["3.2<br/>Entity Extraction<br/><i>clinical terms</i>..."]
  B --> C["3.3<br/>Hybrid Search<br/><i>find 15 candidates</i>"]
  C --> D["3.4<br/>Re-Ranking<br/><i>narrow to top 3</i>"]
  D --> E["3.5<br/>Specificity Check<br/><i>suggest children codes</i>"]
```

Sub-Step 3.1: Context Filter (already done in Step 2)

The patient's age and sex are passed as hard filters so irrelevant codes are never even considered.

Sub-Step 3.2: Entity Extraction (already done in Step 3)

The clinical entities become the search queries.

Sub-Step 3.3: Hybrid Search (Finding Candidates)

What it does

Searches the vector database using two different methods simultaneously and combines the results.

Why two methods?

Method	Good at	Bad at	Example
--------	---------	--------	---------

Dense Search (semantic/meaning)	Understanding paraphrases and syn	Exact acronyms and rare terms	"heart attack" ? finds "Acute myocardial infar
Sparse Search (keyword/BM25)	Exact matches, acronyms, codes	Understanding that two different phr	"COPD" ? finds "COPD" directly

Neither alone is sufficient:

- ? Dense search alone might miss "COPD" (it's an acronym, not a natural phrase)
- ? Sparse search alone would miss "heart attack" ? "myocardial infarction" (different words, same meaning)

How index synonyms help both methods: Because we embed the Index file's natural-language terms (like "Ear infection, middle") alongside the formal description, the sparse/BM25 layer now matches on colloquial keywords too ? not just the formal ICD-10 terminology. And the dense layer benefits from richer context in the embedding text, making the SapBERT vectors more discriminative.

How they're combined

Final Score = (Dense Score × 0.7) + (Sparse Score × 0.3)

Dense gets more weight (0.7) because in clinical text, synonyms and paraphrases are more common than exact acronyms. But sparse (0.3) still matters for those critical exact matches.

The metadata filter

Before searching, the system applies a hard filter:

This means a male patient will never see pregnancy codes in results ? they're filtered out at the database level before scoring even begins.

Output

Top 15 candidates ? intentionally broad. We cast a wide net now and narrow down next.

Sub-Step 3.4: Cross-Encoder Re-Ranking (Precision Layer)

What it does

Takes the 15 candidates from hybrid search and scores each one directly against the transcript, keeping only the top 3.

Why it's needed

Vector search is good at recall (finding things that might be relevant) but mediocre at precision (knowing which ones are actually the best match). The re-ranker fixes this.

How it's different from the vector search

Vector Search (Step 3.3)	Cross-Encoder Re-Ranking (Step 3.4)
Compares embeddings (compressed represen	Reads both texts together, word by word
Fast (~50ms for 72,000 items)	Slow (~200ms for 15 items)
Good at finding the right neighborhood	Good at picking the best within that neighborhood
Like scanning a library catalog	Like reading and comparing 15 book summaries si...

Why not use the cross-encoder for everything?

It's too slow. Scoring 72,000 codes pairwise would take minutes. So we use the fast vector search to narrow to 15, then the precise cross-encoder to narrow to 3.

Example

Transcript: "Patient reports bilateral ear pain, worse at night, fever 101°F"

Candidate Codes (from hybrid search):

1. H65.93 - Otitis media, bilateral (score: 0.91) ? KEPT
2. H66.93 - Suppurative otitis media, bilateral (score: 0.87) ? KEPT
3. H65.9 - Unspecified otitis media (score: 0.82) ? KEPT
4. H60.90 - Otitis externa (score: 0.45) ? DROPPED (external, not middle ear)
- 5?15. Lower-scoring codes ? ALL DROPPED

Sub-Step 3.5: Hierarchical Specificity Check

What it does

Checks if any of the top 3 codes are too general (parent codes) and suggests more specific children codes for the doctor to choose from.

Why it's needed

ICD-10 is hierarchical. Billing with a vague "unspecified" code when more specific information is available leads to:

- ? Claim denials from insurance
- ? Lower reimbursement
- ? Audit flags

Example

```
Top code: H65.9 (Unspecified otitis media)
  ? Has children! Expand:
    ??? H65.91 - Otitis media, right ear
    ??? H65.92 - Otitis media, left ear
    ??? H65.93 - Otitis media, bilateral  ? transcript says "both ears"
```

```
System prompts the doctor:
"The transcript mentions bilateral involvement.
Consider H65.93 (bilateral) for greater specificity."
```

The Zero-Hit Fallback

If no code scores above a confidence threshold (0.7), the system gracefully declines:

"Insufficient data to suggest an accurate ICD-10 code. Please manually assign in EHR."

This prevents the system from forcing a bad suggestion just to fill the field.

Step 5: Note Generation (LLM Call)

What it does

Sends the transcript + RAG results to the main LLM (Claude Sonnet) to generate the structured SOAP note.

Why it's needed

The RAG pipeline found the right codes, but someone needs to actually write the clinical note ? structuring the transcript into Subjective/Objective/Assessment/Plan sections, integrating the ICD-10 suggestions, and framing everything as suggestions (not diagnoses).

What the LLM receives

```
[System Prompt: SOAP format rules, no-diagnosis rules, citation requirements]

[User Message:]
Reference ICD-10 codes (from verified RAG retrieval):
- H65.93: Unspecified otitis media, bilateral (Hierarchy: Diseases of ear ? Otitis media)
  Use additional code: code to identify infectious agent
- I10: Essential hypertension (Hierarchy: Diseases of circulatory system ? Hypertensive diseases)

Transcript:
"Doctor: What brings you in today?
Patient: My kid's ears have been hurting for two days..."

Generate a SOAP note. Append "(Pending Physician Confirmation)" to every suggested code.
```

Note the "Use additional code" line ? this comes from the `use_additional_code` element in the Tabular XML. It tells the LLM (and the doctor) that a companion code may be needed for complete documentation. Similarly, `metadata` is passed through when a code has sequencing requirements.

Why inject RAG results into the prompt?

This is the "Augmented" part of RAG. By giving the LLM the verified codes directly:

- ? It won't hallucinate fake codes (it has real ones to use)
- ? It can cite the source ("Source: ICD-10-CM 2026")
- ? Every code in the output is traceable back to the retrieval step
- ? Sequencing rules (`code_first`, `use_additional_code`) inform proper coding practice

Output

A structured SOAP note with ICD-10 suggestions, all marked as pending physician review.

Step 6: Guardrail Check (Safety Layer)

What it does

Validates the generated note against safety rules before showing it to the doctor.

Why it's needed

Even the best LLMs sometimes:

- ? Slip into definitive diagnosis language ("The patient has pneumonia")
- ? Suggest medication dosages not mentioned in the transcript
- ? Miss dangerous symptom combinations that need urgent escalation

Guardrails are the last line of defense between the LLM's output and the doctor's screen.

The Three Checks

Check 1: Red-Flag Detection (Pre-generation, on transcript)

```
Transcript: "chest pain radiating to left arm, shortness of breath"
? MATCH: Cardiac emergency pattern
? ACTION: Skip routine note, escalate immediately
```

Red-flag patterns:

- ? Chest pain + radiation ? Potential MI
- ? Thunderclap headache + stiff neck ? Potential SAH
- ? Sudden one-sided weakness ? Potential stroke
- ? Acute dyspnea + chest tightness ? Potential PE

Check 2: Diagnosis Assertion Detection (Post-generation, on note)

Note contains: "The patient has acute otitis media"
? MATCH: Definitive diagnosis language
? ACTION: Reframe to "Possible acute otitis media (for physician review)"

Banned patterns:

- ? "The patient has..."
- ? "Diagnosis is..."
- ? "Confirmed diagnosis of..."
- ? "This is clearly..."

Check 3: Dosage Fabrication Check (Post-generation, comparing note vs transcript)

Note contains: "Amoxicillin 500mg TID"
Transcript contains: (nothing about dosages)
? MATCH: Dosage not from transcript
? ACTION: Remove fabricated dosage, flag for doctor

What happens when guardrails trigger?

Trigger	Severity	Action
Red-flag symptoms	? Error	Stop everything, show escalation warning
Diagnosis assertion	? Warning	Reframe language, continue
Dosage fabrication	? Warning	Remove dosage, flag it
All clear	? Pass	Continue to response

Step 7: Tool Execution (Calling External Systems)

What is Tool Calling?

Tool calling is when the LLM decides it needs to take an action rather than just generate text. Instead of answering with words, it says "I need to call this function with these parameters" ? and the system executes that function and returns the result.

Think of it like a doctor's assistant who can not only write notes but also file them in the chart system, look up records, etc.

Why tools are needed here

- The agent needs to interact with the outside world:
- ? Save a note ? Write to the EHR (Electronic Health Record) system
 - ? Look up history ? Read from the EHR system

Without tools, the agent could only generate text. With tools, it can actually do things.

How Tool Calling Works (Step by Step)

```
sequenceDiagram
    participant D as Doctor
    participant A as Agent (LLM)
    participant T as Tool System
    participant E as Mock EHR API

    D->>A: "Save this note to P001's chart"
    A->>A: Decides: I need to call save_note()
    A->>T: Tool Call: save_note(patient_id="P001", note="...", icd_codes=["G44.2"])
    T->>E: POST /notes {patient_id, note, icd_codes}
    E->>T: {status: "saved", note_id: "N_abc12345"}
    T->>A: Result: Note saved with ID N_abc12345
    A->>D: "? Note saved to P001's chart. Note ID: N_abc12345"
```

The Two Tools

save_note ? Write to Chart

```
Purpose: Save a completed clinical note to the patient's EHR
Input:  patient_id (required), note text (required), icd_codes (optional)
Output: {status: "saved", note_id: "N_abc12345", timestamp: "..."}
Errors: Missing patient_id ? 400, Empty note ? 400
```

Critical safety rule: The agent cannot auto-save. It can only call save_note when the doctor explicitly asks to save. This prevents accidental saves of unreviewed notes.

get_patient_history ? Read from Chart

```
Purpose: Retrieve prior visit notes for a patient
Input:  patient_id (required)
Output: {visits: [{note_id, date, summary, icd_codes}, ...]}
Errors: Unknown patient ? {status: "no_history"}
```

MCP (Model Context Protocol)

Tools are exposed to the agent via MCP ? a standard protocol for connecting LLMs to external capabilities. Think of MCP as a universal plug adapter: the tools follow a standard interface, so any LLM that speaks MCP can use them without custom integration.

```
Agent ?? MCP Protocol ?? Tool Server ?? Mock EHR API (FastAPI + JSON file)
```

What happens when the tool fails?

- The system handles errors gracefully:
- ? EHR API down ? "Unable to save to EHR at this time. Please try again or save manually."
 - ? Invalid patient ? "Patient ID not found. Please verify the ID and try again."
 - ? Network error ? Retry once, then report the failure

Step 8: Memory Lookup (Patient Context)

What it does

Queries the SQLite database for prior visit information about this patient.

Why it's needed

Doctors need context. If a patient came in last week with a headache and is back today with the same complaint, that's critical information ? it changes the differential (recurrent vs. new onset).

How it works

```
Input:  patient_id = "P001"
Query:  SELECT * FROM visits WHERE patient_id = 'P001' ORDER BY visit_date DESC
Output: "Prior visits:
        - 2024-03-08: Tension headache, recommended OTC analgesics (G44.2)
        - 2024-02-15: Routine checkup, BP slightly elevated (R03.0)"
```

This context is injected into the note generation prompt so the LLM can reference prior visits appropriately.

Step 9: Response Generation (Final Formatting)

What it does

Takes whatever result came from the preceding steps and formats it into the final response the doctor sees.

Why it's needed

- Different paths produce different kinds of output:
- ? SOAP note ? Format with sections, badges, citations
 - ? ICD lookup ? Format as a code suggestion with source
 - ? Tool result ? Format as a confirmation message
 - ? Escalation ? Format as an urgent warning
 - ? Refusal ? Format as a polite decline with alternative offer

What it adds

1. "(Pending Physician Confirmation)" tag on every ICD-10 code
2. Source citations for every code: (Source: ICD-10-CM 2026)
3. UI badges: Cache hit ?, Guardrail active ??, Escalation ??, Latency ??
4. Trace data for the expandable trace panel (which nodes ran, timing, RAG chunks)

Complete Path Examples

Example 1: SOAP Note Generation (Happy Path)

```
Doctor: "Patient reports headache for 3 days, worse in the morning. BP 130/85."

Step 1 (Parse): Intent = "soap" ? route to full pipeline
Step 2 (Context): Patient P001 ? {age: 34, sex: "female"}
Step 3 (Entities): ? ["Tension-type headache", "Elevated blood pressure"]
Step 4 (RAG):
  - Hybrid Search ? 15 candidates (filtered to female-applicable codes)
  - Re-Rank ? Top 3: G44.2, R51.9, I10
```

```
- Specificity ? G44.2 has no children (already specific)
Step 5 (Generate): Claude produces SOAP note with G44.2 cited
Step 6 (Guardrail): No red flags, no diagnosis assertions ? PASS
Step 9 (Format): Add badges, citations, confirmation tags

? Doctor sees: Structured SOAP note with "G44.2 - Tension-type headache
(Source: ICD-10-CM 2026) (Pending Physician Confirmation)"
```

Example 2: Red-Flag Escalation

```
Doctor: "Patient has chest pain radiating to the left arm, diaphoresis"

Step 1 (Parse): Intent = "soap" ? route to full pipeline
Step 6 (Guardrail ? pre-check on transcript):
- Pattern match: "chest pain" + "radiating" + "left arm" ? RED FLAG
- Severity: ERROR
- Action: STOP ? do not generate routine note

? Doctor sees: "?? URGENT ESCALATION REQUIRED
Red-flag symptom combination detected: Chest pain with radiation ?
potential cardiac emergency.
Recommend immediate in-person emergency evaluation.
Do NOT proceed with routine documentation until the attending physician reviews."
```

Example 3: Tool Call (Save Note)

```
Doctor: "Save this note to patient P001's chart"

Step 1 (Parse): Intent = "save" ? route to tool execution
Step 7 (Tool):
- LLM decides: call save_note(patient_id="P001", note=<last note>, icd_codes=["G44.2"])
- API call: POST http://localhost:8100/notes
- Response: {status: "saved", note_id: "N_abc12345"}
Step 9 (Format): Confirmation message

? Doctor sees: "? Note saved to P001's chart. Note ID: N_abc12345"
```

Example 4: Diagnosis Refusal

```
Doctor: "Diagnose this patient's condition for me"

Step 1 (Parse): Intent = "refuse" ? route directly to response
Step 9 (Format): Refusal message

? Doctor sees: "I cannot provide a definitive diagnosis. As a documentation
assistant, I can offer suggested differentials as decision support only.
The attending physician must make the final diagnostic determination."
```

Why This Architecture? (Design Rationale)

Design Choice	Alternative	Why we chose this
Multi-step pipeline	Single LLM call	Auditability ? each step is logged; if a wrong ...
RAG over pure LLM	Trust LLM memory	LLMs hallucinate medical codes; RAG guarantees ...
Dual-source ETL (Tabular + Index)	Tabular only	The Index file is a free synonym dictionary; en...
Hybrid search	Dense only	Acronyms like "COPD" need exact keyword matchin...
Cross-encoder re-rank	Trust vector scores	Vector similarity ? clinical relevance; the re-...

Guardrails as separate step	Prompt engineering only	Defense in depth ? prompts can be bypassed; reg...
Tools via MCP	Direct API calls	Standard protocol; swappable backends; compatib...
Stubs for future weeks	Build everything at once	Incremental delivery ? each week's demo works e...

Glossary

Term	Meaning
SOAP	Subjective, Objective, Assessment, Plan ? stand...
ICD-10	International Classification of Diseases, 10th ...
ICD-10-CM Tabular	The hierarchical classification file ? defines ...
ICD-10-CM Index	The alphabetical lookup file ? maps natural-lan...
RAG	Retrieval-Augmented Generation ? look up facts ...
Dense Search	Vector similarity search using embeddings (capt...
Sparse Search	Keyword/BM25 search (captures exact terms and a...
Cross-Encoder	A model that reads two texts together to score ...
SapBERT	A medical embedding model trained on UMLS medic...
MCP	Model Context Protocol ? standard for connectin...
EHR	Electronic Health Record ? the digital patient ...
Guardrail	A safety check that runs on generated output be...
LangGraph	A framework for building LLM applications as ex...
ETL	Extract-Transform-Load ? the pipeline that conv...